

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	YAZHINI.E.M	Reg.No.:	192511165
Department:	B.E.CSE	Date:	27/08/2026

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
 - Maximum interrupt latency < 10 μ s
 - Use vectored interrupts for high-priority devices
 - Use software interrupts for background tasks
-

Code of Conduct:

I YAZHINI.E.M/192511165 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1 — Real-Time Embedded I/O Communication Mechanism

Given Constraints

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

Constraint Analysis: The dominant constraint is the sub-15% CPU budget under a 1 kHz interrupt arrival rate (one interrupt per sensor every 1 ms). Any polling-based scheme is immediately ruled out because it would burn CPU cycles even when no sensor has new data. This forces an interrupt-driven design, and because several sensors compete for the same CPU, the ISR dispatch overhead per interrupt must be minimized — which is exactly what vectored interrupts are built for.

Solution: Each sensor is wired to a distinct IRQ line on a hardware interrupt controller. On assertion, the controller automatically indexes into a vector table and jumps directly to the correct Interrupt Service Routine (ISR), eliminating the software polling loop that non-vectored designs need to identify the interrupt source. For the I/O technique, memory-mapped I/O is selected: sensor registers are mapped into the normal address space, so the CPU can use its fast, pipeline-friendly load/store instructions instead of the separate, higher-latency IN/OUT instruction set used by I/O-mapped I/O.

Vectored Interrupt Handling — Real-Time Sensor Network

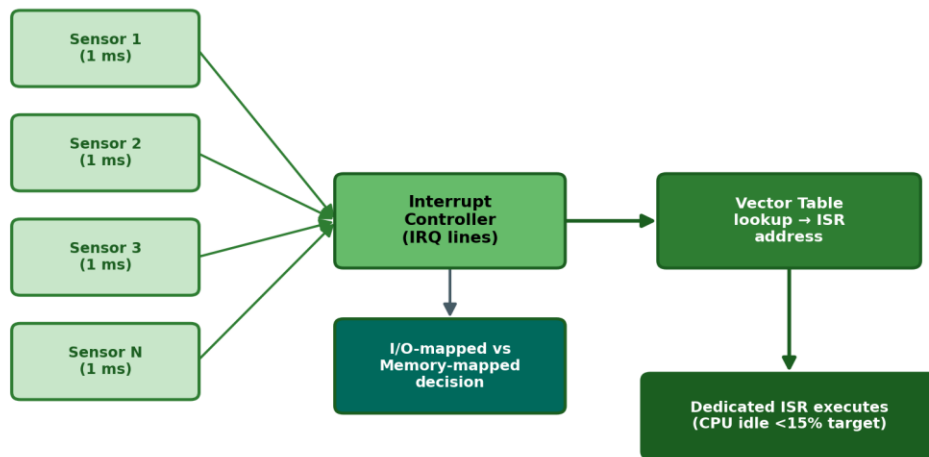


Figure 1: Vectored interrupt dispatch path from sensors to ISR execution

Justification: Vectored dispatch removes the source-identification overhead of a shared, single-vector interrupt line, cutting per-interrupt CPU cost well below the level needed to stay under 15% utilization at a 1 kHz interrupt rate. Memory-mapped I/O further reduces per-access latency versus I/O-mapped I/O because it uses the same fast bus-access path and addressing modes as ordinary memory operations, satisfying the optimal-latency requirement.

Question 2 — High-Speed NVMe Storage I/O Subsystem

Given Constraints

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

Constraint Analysis: The 3 GB/s throughput target immediately eliminates legacy interfaces such as SATA/AHCI, whose queue-depth and command-overhead ceiling caps out far lower. PCIe is the only bus family with enough per-lane bandwidth headroom, and NVMe is the protocol purpose-built to exploit it with deep, parallel command queues. The 'CPU should not handle bulk transfer' constraint is the deciding factor for the data path: it rules out Programmed I/O and mandates DMA.

Solution: The CPU issues only the command (submission-queue entry) to the NVMe controller over the PCIe link; the DMA controller then transfers the actual data payload directly between the SSD and system memory without CPU involvement in the byte-by-byte copy. Once the transfer completes, the DMA engine raises a single completion interrupt so the CPU is only interrupted once per operation rather than once per data chunk.

NVMe-over-PCIe I/O Subsystem with DMA Offload

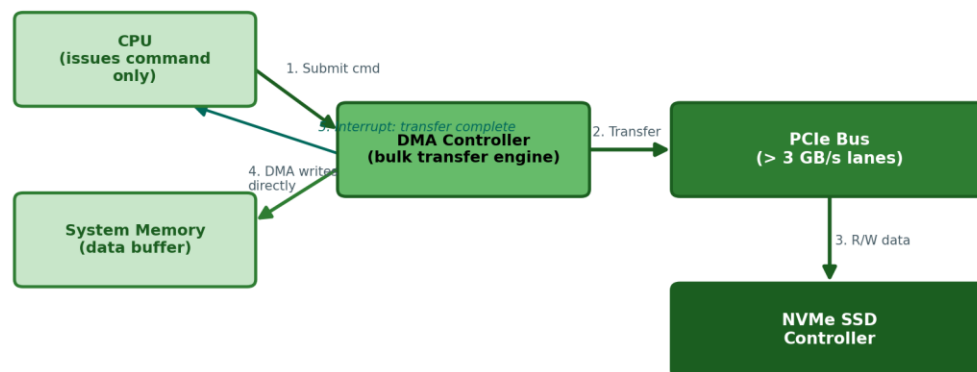


Figure 2: NVMe-over-PCIe data path with DMA offload and completion interrupt

Justification: Offloading bulk transfer to DMA frees the CPU almost entirely from the data path, satisfying the 'no bulk transfer on CPU' constraint, while PCIe's multi-lane, multi-gigabit signalling combined with NVMe's queue depth of up to 64K commands per queue comfortably clears the >3 GB/s throughput requirement. Interrupt-based completion notification avoids wasteful polling of transfer status.

Question 3 — Multi-Device USB / Thunderbolt Communication Architecture

Given Constraints

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts

Given Constraints

- Ensure fair bus arbitration

Constraint Analysis: With 8 total devices sharing host bandwidth across two different bus standards, the central risk is that the 2 high-throughput Thunderbolt devices could dominate the arbiter and starve the 6 USB peripherals of servicing. The design must therefore balance raw priority (Thunderbolt needs low, predictable latency) against fairness (no USB device should be permanently deferred).

Solution: A host-side arbiter combines priority scheduling with round-robin fairness: Thunderbolt traffic is granted a higher base priority tier for its time-sensitive, high-speed transfers, while the 6 USB devices are serviced in round-robin order within their tier. An aging counter raises the effective priority of any USB device that has waited too long, guaranteeing bounded worst-case latency. Time-critical peripherals raise hardware interrupts directly to the controller; background/status-polling tasks (e.g., periodic device-health checks) are handled through software interrupts issued by the driver stack, keeping the hardware IRQ line reserved for genuinely urgent events.

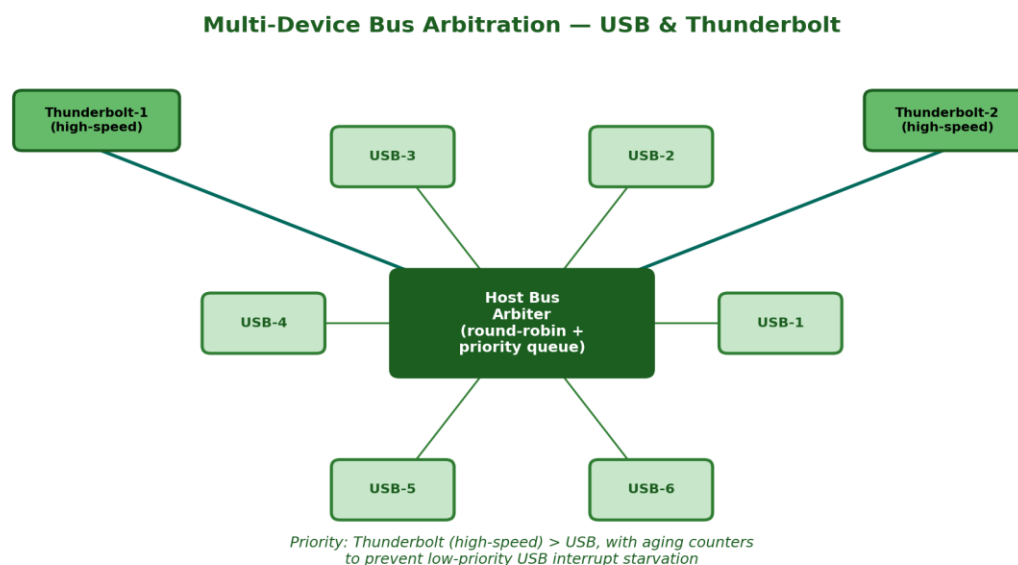


Figure 3: Arbiter servicing USB and Thunderbolt devices with priority + round-robin fairness

Justification: The hybrid priority/round-robin scheme with aging guarantees that no USB device is starved indefinitely, directly satisfying the anti-starvation constraint, while still letting the 2 Thunderbolt devices meet their high-speed timing needs. Separating hardware and software interrupts by urgency keeps the interrupt controller from being overloaded with low-priority housekeeping events.

Question 4 — Continuous Data Acquisition System

Given Constraints

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O

Given Constraints

- Use memory-mapped I/O for device registers

Constraint Analysis: At 500 MB/s of continuous streaming data, a pure interrupt-driven approach — where the CPU is interrupted for every data unit and copies it itself — would consume a very large share of CPU cycles purely on data movement, violating the 'minimal CPU intervention' constraint. This makes DMA the clear choice for the data path, while the device's control/status registers (which are accessed far less frequently) are still exposed via memory-mapped I/O for simple, fast configuration access.

Solution: The acquisition device streams samples directly into system memory through the DMA controller in large, pre-configured block transfers. The CPU is only interrupted once per completed block (not once per sample), and in between blocks it is free to process previously acquired data or perform other tasks. Device configuration registers (sample rate, channel select, start/stop) remain memory-mapped so the CPU can set them up with ordinary load/store instructions before streaming begins.

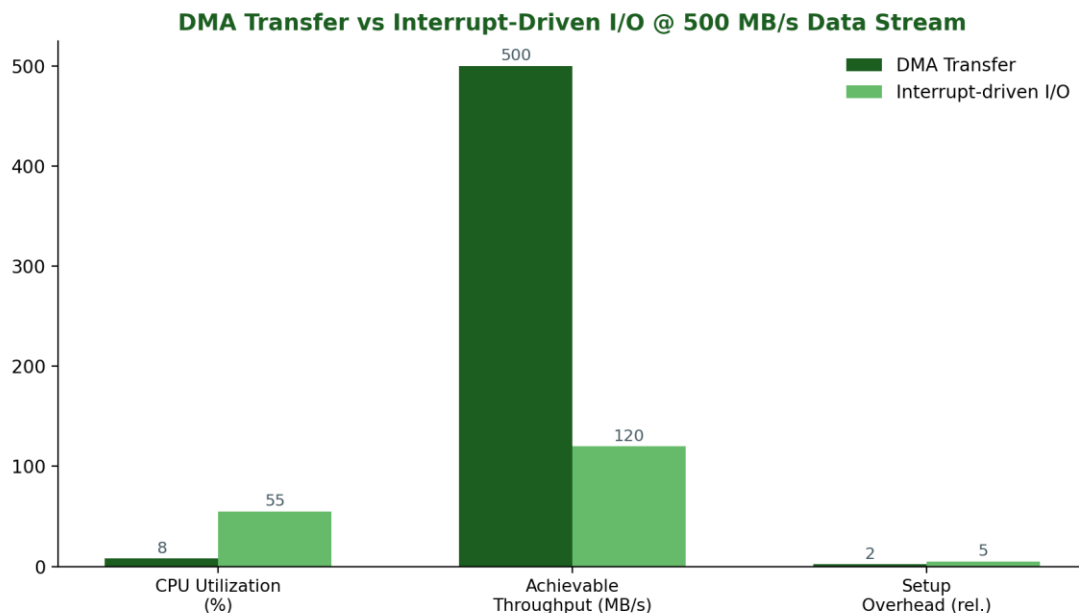


Figure 4: DMA transfer vs. pure interrupt-driven I/O at a 500 MB/s data rate

Justification: As the comparison shows, DMA transfer keeps CPU utilization low and sustains far higher effective throughput than interrupt-driven I/O at the same data rate, because the CPU is removed from the per-sample data path and only handles block-level completion events. This satisfies both the throughput and the minimal-intervention constraints simultaneously, while memory-mapped registers keep control-path latency low.

Question 5 — Nested Interrupt Handling Mechanism

Given Constraints

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Constraint Analysis: A strict $<10\ \mu\text{s}$ latency ceiling means a lower-priority ISR must never be allowed to block a higher-priority event for long, which is precisely what nested interrupt handling addresses: it lets a higher-priority interrupt preempt an ISR that is already executing. The design therefore needs a clear priority hierarchy, not just a flat interrupt queue.

Solution: Interrupt sources are organized into priority levels. Critical, time-sensitive devices use vectored hardware interrupts at the top levels so the controller jumps straight to their ISR with no software dispatch delay. A nested-vector interrupt controller tracks the priority of the currently executing ISR and immediately preempts it if a higher-priority interrupt arrives, resuming the lower-priority ISR afterward. Background, non-time-critical work (logging, housekeeping, deferred processing) is triggered via software interrupts at the lowest priority level, so it never blocks hardware events.

Nested Interrupt Priority Model (Latency $< 10\ \mu\text{s}$)

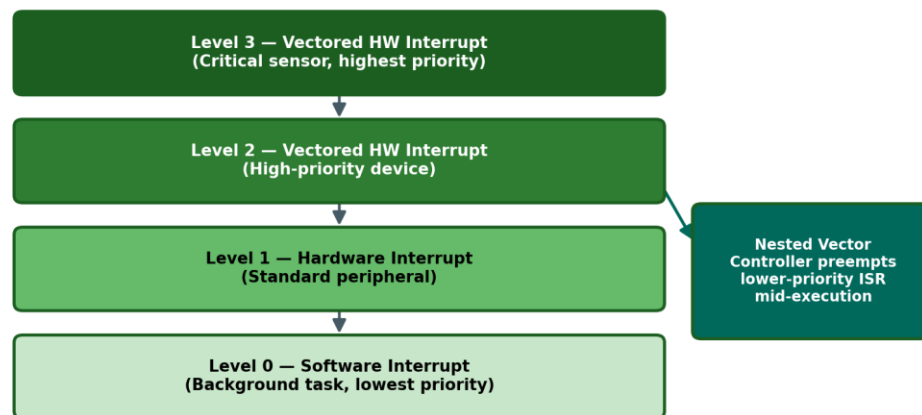


Figure 5: Four-level nested interrupt priority hierarchy with preemption

Justification: Priority-based preemption ensures the highest-priority vectored interrupt is always serviced immediately regardless of what else is running, which is the mechanism needed to bound worst-case latency under $10\ \mu\text{s}$. Reserving software interrupts strictly for background tasks keeps the hardware interrupt path short and predictable, and vectoring removes source-lookup delay from the critical path.

SUMMARY TABLE

S.NO	Core Constraint Driver	Key Design Choice
1	CPU < 15% @ 1 kHz interrupts	Vectored HW interrupts + memory-mapped I/O
2	Throughput > 3 GB/s, no CPU bulk copy	NVMe-over-PCIe + DMA with completion interrupt
3	8 devices, no starvation	Priority + round-robin arbitration with aging
4	500 MB/s, minimal CPU load	Block-mode DMA + memory-mapped control registers
5	Latency < 10 μ s	Nested, preemptive vectored interrupt priority levels